

VIT BHOPAL UNIVERSITY

School of Engineering / ECE

AI-POWERED RF SPECTRUM INTELLIGENCE PLATFORM

Detailed Project Report

A software-defined RF signal analysis and modulation classification platform using digital signal processing and convolutional neural networks

Project Detail	Value
Student	Manthan Sontakke
Program	B.Tech. Electronics and Communication Engineering
Specialization	Artificial Intelligence and Cybernetics
Academic Period	2026-27
Project Type	Final Year / Major Project

Submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology.

CERTIFICATE

This is to certify that the project titled “AI-Powered RF Spectrum Intelligence Platform” has been carried out by Manthan Sontakke, student of B.Tech. Electronics and Communication Engineering (Artificial Intelligence and Cybernetics), VIT Bhopal University, as a major/final-year project. The work described in this report represents the design and implementation carried out as part of the project study.

Project Guide: _____

Programme Chair / Department: _____

Date: _____ Place: _____

DECLARATION

I hereby declare that this project report titled “AI-Powered RF Spectrum Intelligence Platform” is prepared for academic purposes and presents the design, implementation, experiments, and observations associated with the project. The implementation details are based on the project source code, dataset-generation pipeline, AI classification pipeline, testing utilities, and Streamlit demonstration interface developed during the project work.

Student Signature: _____

Name: Manthan Sontakke

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to VIT Bhopal University and the faculty members of the Electronics and Communication Engineering department for providing the academic environment and resources needed to undertake this project. I am grateful to the project guide and faculty members whose feedback helped shape the system architecture, implementation approach, and validation process. I also acknowledge the open-source communities behind Python, NumPy, signal-processing libraries, TensorFlow/Keras, OpenCV, Matplotlib, and Streamlit, which provided the software ecosystem used to build and demonstrate the platform.

ABSTRACT

The AI-Powered RF Spectrum Intelligence Platform is an ECE-oriented signal analysis system designed to combine digital signal processing (DSP), synthetic radio-frequency signal generation, spectrogram-based feature extraction, and deep learning for automatic modulation/signal classification. The platform generates representative communication signals such as BPSK, QPSK, QAM, OFDM, and multi-tone signals; introduces controlled noise and filtering effects; converts signals into visual time-frequency representations; and uses a convolutional neural network (CNN) to learn discriminative patterns from the resulting spectrograms. A modular Python architecture separates signal generation, noise injection, filtering, dataset creation, AI model training, prediction, testing, and visualization. A Streamlit dashboard provides an interactive demonstration in which signal parameters can be varied and the resulting waveforms, frequency-domain characteristics, and AI classification workflow can be inspected. The system is intended as a low-cost research and educational prototype for spectrum monitoring, modulation recognition, and RF signal intelligence. The project also establishes a foundation for a future hardware implementation using an SDR or FPGA-based accelerator, where the trained inference pipeline can be quantized and deployed for real-time operation.

Keywords: RF spectrum intelligence, digital signal processing, modulation classification, spectrogram, convolutional neural network, CNN, SDR, signal analysis, Streamlit, machine learning.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement and Motivation
3. Objectives and Scope
4. Background and Technical Foundations
5. System Requirements
6. Proposed System Architecture
7. Module-wise Design and Implementation
8. AI Dataset and CNN Classification
9. Dashboard and Demonstration
10. Testing and Validation
11. Results and Discussion
12. Limitations and Risk Considerations
13. Future Enhancements
14. Conclusion
15. Project Work Plan

16. References

Appendix A: Repository Structure

Appendix B: Viva-Oriented Summary

LIST OF FIGURES

Figure 1. Overall System Architecture

Figure 2. Development and Evaluation Workflow

LIST OF TABLES

Table 1. Major System Requirements

Table 2. Supported Signal Classes

Table 3. Repository Modules and Responsibilities

Table 4. Testing Strategy

Table 5. Risk and Mitigation Matrix

ABBREVIATIONS

Abbreviation	Meaning
RF	Radio Frequency
DSP	Digital Signal Processing
CNN	Convolutional Neural Network
AI	Artificial Intelligence
ML	Machine Learning
FFT	Fast Fourier Transform
SNR	Signal-to-Noise Ratio
I/Q	In-phase / Quadrature
OFDM	Orthogonal Frequency Division Multiplexing
QPSK	Quadrature Phase Shift Keying
QAM	Quadrature Amplitude Modulation
BPSK	Binary Phase Shift Keying
SDR	Software Defined Radio
UI	User Interface

1. INTRODUCTION

Wireless systems occupy a shared electromagnetic spectrum in which multiple communication technologies operate simultaneously. As the number and diversity of emitters increase, automated identification of signal types becomes useful for spectrum monitoring, interference analysis, communication-system research, and educational laboratories. Traditional analysis based only on manual inspection of waveforms can be slow and difficult when signals overlap or are affected by noise.

This project addresses that challenge by building a modular RF intelligence prototype in software. Instead of depending on a physical RF front-end during initial development, the project generates controlled synthetic signals and reproduces common signal-processing steps in a repeatable way. The resulting data are converted into spectrogram images that act as learned features for a CNN-based classifier. The system therefore connects core ECE subjects - communication systems, signals and systems, DSP, probability/noise concepts, and machine learning - in a single demonstrable platform.

The implementation has been organized as a reusable Python repository. Separate modules handle signal generation, processing, filtering, noise, feature extraction, dataset generation, AI model training, inference, testing, and visualization. A Streamlit application provides a user-facing demonstration layer.

2. PROBLEM STATEMENT AND MOTIVATION

Manual RF signal identification requires knowledge of time-domain behavior, spectral characteristics, modulation formats, and channel effects. When the number of observations becomes large, an automated classification system can reduce repetitive inspection and provide a consistent first-pass decision. A practical student project must also balance accuracy, computational complexity, development time, cost, and explainability.

The project therefore targets the following practical question: how can a low-cost software platform generate representative RF signals, transform them into useful time-frequency features, and automatically identify supported signal classes using an AI model while remaining understandable enough for an engineering demonstration?

3. OBJECTIVES AND SCOPE

3.1 Objectives

- Design a modular RF signal-generation and signal-processing pipeline.
- Support representative communication and signal classes including BPSK, QPSK, QAM, OFDM, and multi-tone signals.
- Introduce controlled noise and filtering to make the data-processing chain closer to practical communication conditions.
- Generate labeled spectrogram images for supervised learning.
- Implement CNN-based signal/modulation classification and prediction utilities.

- Build an interactive Streamlit dashboard for visualization and demonstration.
- Validate the major modules through repeatable scripts and test cases.

3.2 Scope

The current scope is a software-first RF intelligence prototype. It emphasizes repeatability and algorithm development rather than a complete over-the-air receiver. Real SDR hardware, RF mixers, antennas, ADC performance, synchronization loops, carrier recovery, and FPGA deployment are treated as future integration opportunities. The project intentionally avoids storing the generated training dataset and trained binary model inside the Git repository so that the repository remains lightweight; the report describes those artifacts as part of the local development workflow.

4. BACKGROUND AND TECHNICAL FOUNDATIONS

4.1 Digital communication signal classes

BPSK represents binary information through two phase states. QPSK extends phase-shift keying to four possible phase states, allowing two bits per symbol under ideal conditions. QAM varies both amplitude and phase and is commonly represented using constellation points. OFDM divides data across multiple orthogonal subcarriers, producing characteristic time-frequency structure. A multi-tone signal is a useful synthetic test case in which several sinusoidal components appear at different frequencies.

4.2 Noise and filtering

Noise is introduced to study whether the processing and classification chain remains useful under imperfect conditions. Additive white Gaussian noise (AWGN) is a standard controlled model for communication-system experiments. Digital filtering can suppress selected frequency components or improve signal conditioning before feature extraction. The project keeps these operations modular so their effects can be changed without rewriting the rest of the pipeline.

4.3 FFT and spectrogram features

The Fast Fourier Transform (FFT) converts a finite time-domain sequence into a frequency-domain representation and helps reveal dominant spectral components. A spectrogram extends this idea by calculating spectral content over successive windows, creating a time-frequency map. Modulation and waveform families often generate visually distinct patterns in such maps, making spectrograms suitable input representations for CNN-based classifiers.

4.4 CNN-based classification

A CNN can learn local spatial patterns through convolutional filters and combine them across deeper layers. In this project, the spectrogram is treated as an image-like representation. The implemented training and prediction code is located in the ai-model module, while the dataset generator creates labeled images organized by class. The training process uses separate data partitions and produces accuracy/loss plots for inspection.

5. SYSTEM REQUIREMENTS

Category	Requirement
Operating system	Windows / Linux capable of running Python and Streamlit
Language	Python 3.x
Core packages	NumPy, Matplotlib, TensorFlow/Keras, Streamlit and supporting DSP/image utilities
Development IDE	VS Code / PyCharm / any Python-compatible IDE
Hardware	A standard PC/laptop; optional GPU for faster model training
Data storage	Space for generated spectrogram datasets and model checkpoints during experimentation
Version control	Git and GitHub

6. PROPOSED SYSTEM ARCHITECTURE

The platform is organized as a layered pipeline. The lower layers create and process signals; the middle layer converts those signals into machine-learning data; the AI layer trains and evaluates the classifier; and the presentation layer exposes the workflow through the dashboard.

Figure 1. Overall System Architecture

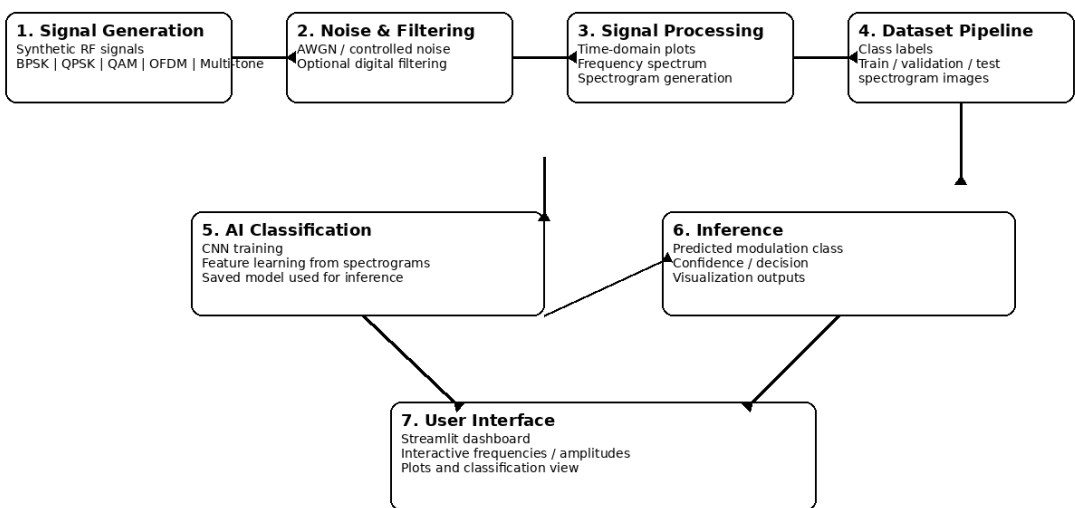


Figure 1. Overall System Architecture

6.1 Data flow

- 1. Select or generate a supported RF/communication signal.
- 2. Apply amplitude, frequency, duration, and sampling parameters.
- 3. Inject noise and optionally apply a digital filter.
- 4. Compute time-domain, frequency-domain, and spectrogram representations.
- 5. Generate labeled image samples and organize them into training, validation, and test sets.
- 6. Train the CNN classifier and evaluate loss/accuracy.
- 7. Use the trained classifier for prediction on new spectrogram data.
- 8. Present outputs through Streamlit and supporting visualizations.

7. MODULE-WISE DESIGN AND IMPLEMENTATION

Module / Path	Responsibility
src/signal_generation/	Creates synthetic communication and RF-like signals.
src/signal_processing/	Performs core DSP operations and transforms.
src/noise/	Adds controlled noise models to signals.
src/filters/	Provides digital filtering and conditioning operations.
src/feature_extraction/	Creates features/representations used by downstream analysis.
src/pipeline/	Connects multiple processing stages into reusable workflows.
src/visualization/	Plots waveforms, spectra and other analysis outputs.
src/communication/	Holds communication-signal specific generation/integration code.
src/datasets/	Creates and manages labeled AI dataset generation.
src/ai_models/	Contains CNN definition, training, and prediction code.
app.py	Streamlit interactive application.
test_prediction.py / test_*.py	Validation and prediction-related tests.
results/	Stores selected training/evaluation figures and metrics summaries.

7.1 Signal generation

The signal-generation stage provides controlled test signals so that the remaining pipeline can be verified without requiring an external transmitter. For example, the project can produce single-tone or multi-tone waveforms and communication signals such as BPSK, QPSK, QAM, and OFDM. Parameterized generation makes it possible to vary carrier/frequency, amplitude, sampling rate, and duration.

7.2 Noise and filtering

Noise is added as a separate stage to make the experiments repeatable. Filtering is then applied as required by the selected workflow. Keeping both operations independent enables controlled comparisons and helps isolate the effect of each processing block during debugging and evaluation.

7.3 Dataset generation

The project includes a dedicated dataset-generation path that converts processed signals into spectrogram images and assigns the correct class label. The local dataset is arranged by class and split for model development. Because these images are generated artifacts rather than source code, they are excluded from the Git repository through the project .gitignore configuration.

7.4 Visualization

Visualization utilities show the signal in multiple representations. A typical analysis view includes a time-domain waveform, a frequency spectrum, and a spectrogram. These views help bridge signal-processing theory and machine-learning behavior: a user can see what the signal looks like physically and then see how the model receives an image-like representation.

8. AI DATASET AND CNN CLASSIFICATION

8.1 Supported classes

Class	Engineering significance
BPSK	Binary phase-shift keyed communication waveform
QPSK	Four-state phase-shift keyed waveform
QAM	Amplitude-and-phase varying digital modulation
OFDM	Multi-subcarrier communication waveform
MULTI_TONE	Synthetic multi-frequency test signal

8.2 Dataset organization

The AI dataset uses a directory-per-class structure that is compatible with common image-loading pipelines. The repository development workflow includes train, validation, and test partitions.

Generated spectrograms are stored locally and can be recreated using the dataset-generation code, which improves reproducibility without inflating the Git repository.

8.3 Model pipeline

The CNN pipeline consists of data loading, image preprocessing, model construction, supervised training, validation, metric visualization, and prediction. The model definition is kept separate from training and prediction so that different network architectures can be explored without changing the rest of the platform.

8.4 Why spectrograms are useful

Different modulation and waveform families produce different patterns across time and frequency. A CNN is well suited to discovering such local structures automatically. This approach also provides a visually interpretable intermediate representation: users can inspect the spectrogram even when the final prediction is generated automatically.

9. DASHBOARD AND DEMONSTRATION

The Streamlit application is the primary interactive demonstration interface. It allows the user to vary signal parameters and observe the generated results without needing to execute each processing module manually from a command line.

9.1 Dashboard functions

- Interactive control of signal frequencies and amplitudes.
- Visualization of signal waveform and spectrum.
- Spectrogram generation for AI-oriented analysis.
- Display of classification/prediction results where the trained model is available.
- Rapid experimentation with different input combinations.

9.2 Demonstration sequence

1. Launch the Streamlit application.
2. Select a supported signal configuration from the sidebar controls.
3. Generate the signal and inspect the waveform.
4. Observe its frequency-domain behavior and spectrogram.
5. Run the AI prediction pipeline and note the predicted class.
6. Change the signal parameters and repeat to test sensitivity.

Figure 2. Development and Evaluation Workflow

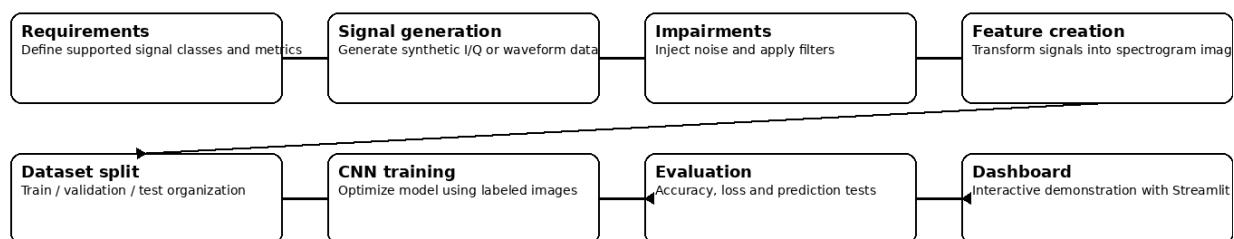


Figure 2. Development and Evaluation Workflow

10. TESTING AND VALIDATION

Test Area	Validation Method	Expected Outcome
Signal generation	Generate each supported class	Valid waveform returned without runtime error
FFT / visualization	Run spectrum calculation on sample signals	Dominant components visible at expected frequencies
Noise module	Apply controlled noise to a baseline signal	Waveform changes while retaining valid data shape
Dataset generator	Create labeled spectrogram samples	Images created under the correct class folders
CNN training	Train on train partition and validate	Loss/accuracy curves generated and training completes
Prediction	Run model on held-out/sample image	Valid class label returned
Dashboard	Launch Streamlit and interact with controls	Plots render and parameter changes update outputs
Repository integrity	Run tests and inspect project tree	Core source code remains separated from generated artifacts

10.1 Test scripts

The development history includes focused scripts for pipeline and prediction validation. These scripts support iterative debugging and reduce the chance that a change to one module silently breaks another. The tests are especially useful when the project is extended with additional modulation classes or a new model architecture.

11. RESULTS AND DISCUSSION

The implemented platform successfully establishes an end-to-end flow from signal generation to interactive AI-oriented analysis. During development, the project progressed from basic waveform generation to multi-tone signal generation, noise/filter integration, spectrogram dataset creation, CNN training, prediction testing, and Streamlit visualization.

11.1 Functional outcomes

- The signal-generation framework supports multiple signal families used for RF/communication experiments.
- The processing pipeline can produce waveform, spectrum, and time-frequency views for analysis.
- The AI dataset generator creates class-labeled spectrogram images suitable for supervised CNN training.
- The CNN training and prediction modules are separated into reusable source files.
- The Streamlit dashboard provides a practical demonstration environment for the complete workflow.
- The repository was reorganized so large generated datasets and trained model binaries are kept out of Git history, while source code, selected results, and the demo video remain suitable for GitHub hosting.

11.2 Interpretation of results

The project demonstrates the feasibility of combining classical DSP with deep learning for automated signal identification. The strongest engineering value of the prototype is not only the classifier itself but the complete pipeline surrounding it: controlled data generation, signal impairment modeling, feature construction, AI training, testing, visualization, and reproducible software organization. This makes the system suitable for further work toward real-time RF monitoring.

11.3 Performance metrics to record in the final experiment

For the final academic evaluation, the following values should be recorded from the last stable training run and from the target demonstration environment: training accuracy, validation accuracy, test accuracy, training/validation loss, inference latency per sample, model size, dataset size per class, and robustness under selected noise levels. These metrics provide a stronger quantitative basis for comparing future models or hardware implementations.

12. LIMITATIONS AND RISK CONSIDERATIONS

Issue	Current status	Mitigation / next step
Synthetic data bias	Training data are generated programmatically rather than captured over the air.	Add real SDR captures across multiple environments.
Limited class set	Current model targets a defined set of classes.	Expand the label space gradually and rebalance the dataset.
Noise realism	The current workflow uses controlled digital impairments.	Add fading, frequency offset, phase offset, multipath and hardware impairments.
Compute cost	CNN training can require significant CPU/GPU time.	Use transfer learning, smaller networks, quantization and batch optimization.
Real-time operation	The present platform is primarily software-based.	Integrate an SDR or FPGA inference accelerator.
Generalization	A classifier trained on synthetic data may not generalize equally to unseen emitters.	Evaluate on independent field recordings and unseen parameter combinations.

13. FUTURE ENHANCEMENTS

- Connect the platform to a low-cost software-defined radio front end for real-time capture.
- Introduce realistic channel effects such as multipath fading, carrier-frequency offset, timing offset, and phase noise.
- Add more modulation classes and unknown-signal/open-set detection.
- Use model compression, quantization, pruning, and knowledge distillation for edge deployment.
- Port the inference engine to FPGA/SoC hardware for low-latency operation.
- Add spectrum occupancy estimation, interference detection, and signal-segmentation features.
- Create an experiment manager that automatically sweeps parameter ranges and produces comparison reports.
- Add explainable-AI views such as class-activation maps to show which regions of a spectrogram influenced the prediction.

14. CONCLUSION

The AI-Powered RF Spectrum Intelligence Platform integrates fundamental ECE concepts with modern AI methods in a single modular application. The platform generates representative RF and communication signals, applies controlled impairments and digital processing, converts signals into spectrogram-based data, and uses a CNN for automated classification. The Streamlit dashboard turns the processing chain into an accessible engineering demonstration, while the modular repository structure makes the system easy to extend and test.

As a final-year project, the system provides a clear technical progression: signal theory -> DSP implementation -> dataset engineering -> machine learning -> software integration -> validation -> demonstration. The next major step is to replace purely synthetic inputs with SDR captures and, where required, move computationally intensive stages toward FPGA/edge hardware for real-time RF spectrum intelligence.

15. PROJECT WORK PLAN

Phase	Major activities	Deliverables
Phase 1	Literature/requirement study and repository setup	System requirements and modular code structure
Phase 2	Signal generation	BPSK/QPSK/QAM/OFDM/multi-tone generation modules
Phase 3	DSP pipeline	Noise, filtering, FFT and visualization
Phase 4	Dataset generation	Labeled spectrogram dataset pipeline
Phase 5	AI model	CNN training, validation and prediction
Phase 6	Dashboard	Interactive Streamlit application
Phase 7	Testing	Prediction/pipeline tests and debugging
Phase 8	Documentation	Results, report, demo video and GitHub repository

16. REFERENCES

1. A. V. Oppenheim and R. W. Schaffer, Discrete-Time Signal Processing, Pearson.
2. Ian Goodfellow, Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press.

3. TensorFlow / Keras documentation - model construction and training APIs.
4. Streamlit documentation - Python framework for interactive data applications.
5. Matplotlib documentation - plotting and visualization for scientific Python.
6. NumPy documentation - numerical computing and array-based signal processing support.

APPENDIX A: REPOSITORY STRUCTURE

The cleaned GitHub repository is organized so that source code and demonstration artifacts remain version-controlled while large generated training data and model binaries remain local. A representative structure is:

```
AI_RF_Spectrum_Intelligence/  
|-- app.py  
|-- main.py  
|-- generate_ai_dataset.py  
|-- test_prediction.py  
|-- results/  
|   |-- accuracy_plot.png  
|   |-- loss_plot.png  
|   `-- model_metrics.txt  
|-- src/  
|   |-- ai_models/  
|   |-- communication/  
|   |-- datasets/  
|   |-- feature_extraction/  
|   |-- filters/  
|   |-- noise/  
|   |-- pipeline/  
|   |-- signal_generation/  
|   |-- signal_processing/  
|   `-- visualization/  
|-- Demo video/  
|   `-- AI_RF_demo_video_LinkedIn.mp4  
|-- requirements.txt  
|-- README.md  
`-- .gitignore
```

Local/generated (intentionally ignored by Git):

```
-- ai_dataset/  
`-- trained_models/
```

APPENDIX B: VIVA-ORIENTED SUMMARY

One-minute explanation

This project is an AI-assisted RF spectrum intelligence platform. It generates communication signals such as BPSK, QPSK, QAM and OFDM, adds controlled noise, converts the processed signals

into spectrograms, and trains a CNN to classify the signal type. A Streamlit dashboard lets the user change signal parameters and visually inspect the waveform, spectrum and spectrogram. The software architecture is modular, so the same framework can later be connected to an SDR and an FPGA accelerator for real-time spectrum monitoring.

Why this is an ECE project

The project uses communication-system principles, modulation techniques, Fourier analysis, filtering, noise modeling, time-frequency analysis, and signal-processing pipelines. AI is used as the classification layer rather than replacing the underlying ECE signal-processing concepts.

Most important technical contributions

- Modular RF signal and DSP pipeline.
- Synthetic spectrogram dataset generation for multiple signal classes.
- CNN-based automated classification and prediction.
- Interactive Streamlit demonstration.
- Reproducible testing and GitHub-oriented project organization.
- Clear path toward real SDR and FPGA implementation.